Laboratorio 10: Captura el Router

Ciencias de la Computación VIII
Redes y Protocolos

Javier Rivera — Carné: 18007794

Erick Mejia — Carné: 20006104

20 de octubre de 2026

1. Objetivo y alcance de la entrega

Preparar la captura de tráfico del router WEP dispuesto por el docente y la recuperación estadística de su clave, sin diccionarios. Se documentan comandos, automatización mínima, evidencia reproducible y condiciones adversarias que pueden reducir la calidad de captura.

Estado de este informe: preparación previa a la competencia. Se ejecutó una recuperación real sobre una captura pública de Aircrack-ng y se verificó el ayudante local. El router y el adaptador físico se probarán en clase. No se presentan resultados de captura en vivo ni acciones contra compañeros como si ya hubieran ocurrido. El enunciado establece evaluación individual; la portada incluye los dos nombres solicitados para esta preparación.

Fundamento técnico

WEP utiliza RC4 con una clave secreta compartida y un vector de inicialización (IV) de 24 bits que viaja con cada trama. El espacio limitado de IVs y las debilidades estadísticas de la construcción permiten inferir la clave a partir de tráfico suficiente. PTW aprovecha correlaciones del flujo RC4; no necesita adivinar palabras de una contraseña. Una clave WEP de "64 bits" incluye 40 bits secretos más el IV de 24 bits; la de "128 bits" incluye 104 bits secretos. El trabajo original de PTW sustenta esta recuperación estadística [1].

Aircrack-ng usa PTW por defecto para WEP y permite seleccionar WEP con `-a 1`. Se capturan paquetes completos para conservar el material útil, incluido ARP; no se usa `-w` ni una captura limitada con `-ivs`. No se confunde esta práctica con la obtención de un handshake WPA [2].

Entregables preparados

`informe.pdf`; ayudante `lab10.py`; doce pruebas en `test_lab10.py`; muestra pública `muestras/ptw.cap`; registros locales en `evidencias/`; plantilla de medición para clase. Este documento debe subirse a GES antes de la evaluación, después de completar los carnés.

2. Herramientas, equipo y preparación

Herramienta	Función en el laboratorio
`airmon-ng`	Detectar interfaces y activar/restaurar modo monitor.
`airodump-ng`	Guardar tráfico 802.11 y observar IVs/contadores.
`aireplay-ng`	Prueba activa de inyección y reinyección ARP opcionales.
`aircrack-ng`	Recuperación offline de la clave WEP.
`iw`, Python 3	Revisar hardware e interfaces; ejecutar el ayudante.
Wireshark (opcional)	Examinar tramas, reintentos y gestión 802.11.

En Ubuntu/Debian se instalan las cuatro herramientas Aircrack desde un paquete:

```
sudo apt update
sudo apt install aircrack-ng iw python3
# Opcional para analizar visualmente las tramas:
sudo apt install wireshark
```

Se requiere una **interfaz Wi-Fi física compatible con modo monitor**; para la ruta activa también debe admitir inyección. En una VM hay que asignarle el adaptador USB directamente. Una interfaz Ethernet virtual con Internet no proporciona acceso a tramas 802.11 del aire.

El entorno de preparación fue Ubuntu sobre WSL2, kernel `6.18.54.1-microsoft-standard-WSL2`. Al verificar estaban instalados Aircrack-ng 1.7 e `iw`, pero no se detectó interfaz Wi-Fi. Por ello aquí se puede analizar una captura guardada, pero no se ha validado radio. USB/IP permite conectar dispositivos USB a WSL2; no garantiza que el chipset y su controlador admitan monitor e inyección [7].

Comprobaciones que sí pueden hacerse sin el aula

Desde el directorio del laboratorio:

```
python3 lab10.py comprobar
python3 -m unittest -v test_lab10.py
sha256sum -c evidencias/muestra.sha256
aircrack-ng -a 1 -b 00:12:BF:12:32:29 muestras/ptw.cap
```

La muestra se incluye para practicar sin Internet. Su clave pública esperada es `1F:1F:1F:1F:1F`, según el README oficial [6]. No es la del aula. `comprobar` devuelve 1 si faltan herramientas o no detecta Wi-Fi; ese resultado es esperado en este WSL2. Las pruebas locales no necesitan privilegios de administrador; la prueba de recuperación real se omite explícitamente si Aircrack-ng no está instalado.

3. Procedimiento de captura y recuperación en clase

1. Identificar únicamente el objetivo autorizado

Solicitar al docente SSID, BSSID y canal. No basta con el nombre de la red: el BSSID identifica el AP. Los valores siguientes son ejemplos que se deben **sustituir por los reales**; el BSSID ilustrativo no corresponde al aula.

```
IFACE=wlan0
MON=wlan0mon
BSSID=02:11:22:33:44:55
CANAL=6
sudo airmon-ng
iw list
sudo airmon-ng start "$IFACE" "$CANAL"
iw dev
```

En `iw list` se busca soporte de monitor; luego se comprueba el tipo de interfaz en `iw dev`. Ajustar `MON` al nombre que realmente crea `airmon-ng` [3]. Si el docente no facilita el canal, observar pasivamente con `sudo airodump-ng "$MON"`, identificar el AP autorizado y detener esa observación antes de iniciar la captura con canal fijo.

2. Capturar paquetes completos

Mantener fija la sintonía evita perder tiempo escuchando otros canales.

```
mkdir -p capturas
sudo airodump-ng --bssid "$BSSID" --channel "$CANAL" \
  --encrypt WEP --write capturas/clase \
  --output-format pcap,csv "$MON"
```

Confirmar que el AP muestra WEP y que aumenta `#Data`. Se generan `clase-01.cap` y `clase-01.csv`; si el prefijo ya existe, el sufijo puede ser otro. Conservar la ruta efectivamente creada [4].

3. Recuperar la clave en otra terminal

Volver a definir el BSSID real en esa terminal y seleccionar la captura creada:

```
BSSID=02:11:22:33:44:55
aircrack-ng -a 1 -b "$BSSID" \
  -l capturas/clave-clase.txt capturas/clase-01.cap
```

Se puede analizar mientras continúa la captura. Un resultado insuficiente exige más tráfico útil o revisar la captura, no un diccionario. La cantidad de IVs y el tiempo de éxito dependen de las condiciones; no se promete un umbral. Registrar `KEY FOUND!`, el BSSID y el archivo utilizado, y confirmar el resultado con el docente [2]. La salida hexadecimal es una clave WEP; no tiene por qué corresponder a una frase legible.

4. Automatización, ruta activa opcional y restauración

Ayudante desarrollado

`lab10.py` automatiza el ciclo completo: diagnóstico (`comprobar`), gestión del modo monitor (`monitor`), escaneo de APs WEP (`escanear`), captura dirigida (`capturar`), generación de IVs (`inyectar`, con modos `arpreplay`, `chopchop`, `fragment` y `fakeauth`), desautenticación de clientes del AP (`deauth`), aislamiento de un host por ARP spoofing (`bloquear`) y recuperación offline (`recuperar`). Valida BSSID, canal, interfaz y direcciones IPv4, ejecuta argumentos sin un intérprete de shell, conserva los errores de la herramienta y rechaza una salida de clave ya existente. No implementa criptografía propia; todos los subcomandos activos aceptan `-simular`, que valida y muestra el comando sin ejecutarlo ni transmitir. Equivalentes a los pasos manuales anteriores:

```
# Se pueden revisar los argumentos sin radio ni privilegios:
python3 lab10.py capturar --interfaz "$MON" \
  --bssid "$BSSID" --canal "$CANAL" --simular
# Terminal 1: captura real, con interfaz monitor ya preparada:
sudo python3 lab10.py capturar --interfaz "$MON" \
  --bssid "$BSSID" --canal "$CANAL" --salida capturas/clase
# Terminal 2: definir BSSID de nuevo y usar la captura real:
python3 lab10.py recuperar capturas/clase-01.cap \
  --bssid "$BSSID" --clave capturas/clave-clase.txt
```

Si falta tráfico útil: actividad dirigida al AP del laboratorio

Esta ruta es activa, separada de la captura pasiva. Solo se usará dentro del alcance establecido por el docente. La prueba de inyección envía probes, incluidos probes broadcast aun al indicar un AP; un resultado de cero respuestas no demuestra por sí solo incompatibilidad del adaptador [5].

```
SSID='SSID indicado por el docente'
sudo aireplay-ng --test -e "$SSID" -a "$BSSID" "$MON"
```

Si el docente permite reinyección ARP en el AP de pruebas, se puede usar la MAC de un **cliente propio o expresamente habilitado y ya asociado**. Mantener `airodump-ng` capturando; detener la reinyección con Ctrl+C cuando proceda.

```
CLIENTE=02:AA:BB:CC:DD:EE  # Sustituir por el cliente habilitado
sudo aireplay-ng --arpreplay -b "$BSSID" -h "$CLIENTE" "$MON"
```

La herramienta necesita observar un ARP válido: repetirlo induce al AP a producir nuevas tramas con IVs nuevos. Sin cliente asociado, `inyectar -modo fragment` o `chopchop` obtiene keystream para forjar un ARP, de modo que la generación de IVs deja de depender del tráfico ajeno [8]. La desautenticación (`deauth`) y el aislamiento por ARP (`bloquear`) sí están automatizados y se reservan al entorno aislado autorizado por el docente, según se detalla en la sección siguiente.

Restaurar el adaptador

Detener captura y toda prueba activa con Ctrl+C; después ejecutar `sudo airmon-ng stop "$MON"`. Si se desactivó previamente la gestión del adaptador con NetworkManager, reactivarla. No se ejecuta `airmon-ng check kill` como paso automático: puede interrumpir la propia conectividad. Ante cambios de canal, revisar primero `sudo airmon-ng check` y la gestión del adaptador [3].

5. Condiciones adversarias y ventaja competitiva

La sección adversarial del enunciado pide investigar congestión, pérdida y disponibilidad. El docente precisó el alcance: la competencia se desarrolla sobre una **red inalámbrica aislada** en la que solo participan los integrantes del laboratorio, y autorizó expresamente acciones que degraden la eficiencia de captura de los rivales. Dentro de ese alcance se desarrollaron y validaron en `-simular` las acciones descritas abajo; su ejecución con radio se realiza en el aula, sobre el AP de la práctica y nunca fuera de esa red aislada.

Condición investigada	Efecto esperado sobre la captura	Cómo observarla
Congestión del canal por exceso de tráfico	Más contención y retransmisiones; puede bajar la tasa de IVs únicos recibidos. Más paquetes totales no implica siempre más información criptográfica.	Comparar IVs/s, RXQ y Lost con canal fijo; revisar reintentos en Wireshark.
Interferencia, obstáculos o mala ubicación	Menor recepción, tramas perdidas; pueden verse beacons sin datos suficientes.	Comparar PWR y pérdida al mover solo el propio receptor; el ruido no siempre es visible en una captura.
Abuso de desautenticación o desasociación	Desconecta clientes y puede reducir el tráfico útil. El receptor en monitor puede seguir viendo gestión: no significa que desaparezcan todas las tramas.	Buscar gestión repetida y cambios en estaciones; correlacionar con caída de IVs/s.
Interferencia deliberada de radio	Puede degradar disponibilidad a más de un AP y también perjudicar al propio capturador.	Una captura 802.11 por sí sola no confirma interferencia no 802.11; puede requerir instrumentación de espectro.

Estas relaciones son hipótesis de diagnóstico, no resultados medidos en el aula. `RXQ` resume recepción reciente; `Lost` estima huecos de secuencia por estación; `#Data` cuenta IVs únicos para WEP. Ninguno identifica por sí solo al causante [4]. Wireshark permite filtrar reintentos o gestión del AP autorizado sin transmitir [9]:

```
wlan.bssid == 02:11:22:33:44:55 && wlan.fc.retry == 1
wlan.bssid == 02:11:22:33:44:55 &&
  (wlan.fc.type_subtype == 0x0c || wlan.fc.type_subtype == 0x0a)
```

Acciones desarrolladas para obtener ventaja

Autosuficiencia de IVs. `fakeauth` asocia el adaptador al AP; `fragment` o `chopchop` obtienen keystream aun con la red casi vacía y `arpreplay` reinyecta un ARP forjado para producir miles de IVs sin depender del tráfico de otros. Así la recuperación no se estanca por falta de clientes. **Degradación del rival.** `deauth` desautentica a los clientes del AP: quien captura de forma pasiva se queda sin IVs, mientras el atacante sigue generando los suyos por inyección (que no requiere clientes). `bloquear` realiza ARP spoofing con sockets raw, sin dependencias externas, para aislar a un host tras recuperar la clave; al no habilitar reenvío de IP su tráfico se descarta, y Ctrl+C restaura la caché ARP. Ambos se ejecutan únicamente en la red aislada autorizada. **Física.** Ubicar el propio adaptador donde mejore la recepción, con el cable USB estable. **Mitigación (lo que cierra estos ataques).** Todas funcionan porque WEP carece de integridad y de protección de tramas de gestión: migrar a WPA2/WPA3 con 802.11w neutraliza tanto la recuperación de clave como la desautenticación, y la inspección dinámica de ARP junto con entradas estáticas detiene el *spoofing*. No se comparten procedimientos durante la prueba.

6. Evidencia técnica obtenida y pendientes

Prueba offline ejecutada el 20 de octubre de 2026

Se descargó `ptw.cap` desde el servidor oficial [6]. Se ejecutó el ayudante con el BSSID de la muestra y se guardó la salida completa en `evidencias/recuperacion_muestra.txt`; se eliminaron únicamente secuencias ANSI de presentación del terminal. El registro contiene comando, fecha y código de salida. Extracto del resultado real:

```
python3 lab10.py recuperar muestras/ptw.cap \
  --bssid 00:12:BF:12:32:29 --clave evidencias/clave_muestra.txt
Read 65282 packets.
Starting PTW attack with 30566 ivs.
KEY FOUND! [ 1F:1F:1F:1F:1F ]
Decrypted correctly: 100%
Codigo de salida: 0
```

El resultado coincide con la clave publicada. No se proporcionó esa clave al programa: solo se comparó después. El archivo de salida contiene `1F1F1F1F1F`. El SHA-256 de la captura, conservado en `evidencias/muestra.sha256`, es:

```
a1a442ca2e8062901ba3fc0dfd58f9f412e14c22332ee2e4698eeffab6bd787c
```

La prueba demuestra que funciona el análisis offline con esta muestra; **no demuestra soporte monitor/inyección ni recuperación del router del aula**.

Verificación del ayudante

`python3 -m unittest -v test_lab10.py` finalizó con doce pruebas aprobadas. Comprueban: rechazo de parámetros inválidos (radio, cantidad de ráfagas y direcciones IPv4); simulación sin ejecutar herramientas ni crear salidas; captura inexistente y conservación de clave existente; paso literal de rutas con espacios y propagación de fallo; construcción correcta de los comandos de `monitor`, `escanear`, `inyectar`, `deauth` y `bloquear`; y recuperación real de la clave pública. El registro es `evidencias/pruebas_locales.txt`. El diagnóstico local detectó la suite e `iw`, pero ninguna interfaz Wi-Fi; se conserva su salida con código 1 en `evidencias/diagnostico.txt`.

Registro pendiente para clase

Completar `evidencias/REGISTRO_CLASE.md` con BSSID, SSID y canal autorizados, adaptador/controlador, inicio y fin, captura utilizada, IVs, RXQ, Lost y resultado validado por el docente. Conservar pantalla de monitor, captura WEP y resultado de recuperación, además del SHA-256 del archivo original.

Para comparar condiciones, tomar una línea base de 60 segundos, una ventana controlada de igual duración solo si el docente la dispone y un retorno a la línea base. Mantener constantes equipo, BSSID y canal. Calcular $r_{IV} = (\#Data_{final} - \#Data_{inicial})/\Delta t$. Anotar N/D si falta una métrica; no sustituirla por cero ni inventar resultados.

7. Diagnóstico y conclusiones

Síntoma	Revisión y siguiente paso
No aparece Wi-Fi	Revisar acceso físico USB y controlador. En WSL2, Ethernet virtual no basta; elegir un entorno con el adaptador expuesto.
Monitor no compatible	Revisar modos de `iw list` y el chipset/controlador; la instalación de la suite no modifica las capacidades de radio.
Aviso de canal incorrecto	Verificar BSSID, canal, interfaz monitor y procesos que cambien la sintonía; evitar otra captura saltando canales en el mismo radio.
Beacons pero pocos IVs	Confirmar WEP, tráfico de datos y posición del receptor. Un AP inactivo no proporciona automáticamente material suficiente.
No se recupera la clave	Conservar captura completa, reunir más tráfico útil, verificar BSSID y revisar ARP/PTW; no usar diccionarios como sustituto.
Salida de clave ya existente	Usar otra ruta con `-clave`; el ayudante rechaza sobrescribirla. Con comandos manuales, `-l` sí puede sobrescribir.

La recuperación de la muestra confirma experimentalmente la debilidad de WEP sin diccionario. La ejecución en el aula dependerá del adaptador, el tráfico disponible y el alcance fijado por el docente. Congestión y pérdida pueden retrasar la obtención de IVs útiles; los contadores orientan la investigación, pero no prueban por sí solos un ataque. Para redes de uso real se debe sustituir WEP por un mecanismo de seguridad vigente.

Referencias

Enunciado local: `Laboratorio__10.pdf`, una página. Fuentes primarias consultadas el 20 de octubre de 2026:

1. Tews, Weinmann y Pyshkin. *Breaking 104 bit WEP in less than 60 seconds*. IACR, 2007. https://eprint.iacr.org/2007/120
2. Aircrack-ng: recuperación WEP y PTW. https://aircrack-ng.org/doku.php?id=aircrack-ng
3. Aircrack-ng: Airmon-ng. https://aircrack-ng.org/doku.php?id=airmon-ng
4. Aircrack-ng: Airodump-ng y contadores. https://aircrack-ng.org/doku.php?id=airodump-ng
5. Aircrack-ng: prueba de inyección. https://aircrack-ng.org/doku.php?id=injection_test
6. Aircrack-ng: README y muestra pública. https://github.com/aircrack-ng/aircrack-ng/blob/master/README; https://dl.aircrack-ng.org/ptw.cap
7. Microsoft Learn: conexión USB en WSL. https://learn.microsoft.com/en-us/windows/wsl/connect-usb
8. Aircrack-ng: reinyección ARP. https://www.aircrack-ng.org/doku.php?id=arp-request_reinjection
9. Wireshark: referencia de campos IEEE 802.11. https://www.wireshark.org/docs/dfref/w/wlan.html